Contents

1 People **1**

2 Code **1**
2.1 Theory . . . 2
2.1.1 Notation . . . 2
2.1.2 General . . . 3
2.1.3 Nonlinearity . . . 4
2.1.4 Tank . . . 4
2.1.5 Plumbing/injector . . . 5
2.1.6 Chamber/nozzle . . . 5
2.1.7 Injector-nozzle balance . . . 5
2.2 The CEA rocket solver . . . 6
2.3 My own set of inputs . . . 7

1 People

Avionics:

- Nick (also propulsion)

Payload:

- Mica (contact about Pilotix)

Propulsion:

- Francesco (adapters)
- Robert (parametric)
- Nils (WhatsApp guy, propulsion not-lead)
- David (not-fuel grain guy)

Airbreak:

- Iheb Bouakira (CFD)

Python RocketCEA

2 Code

Input variables:

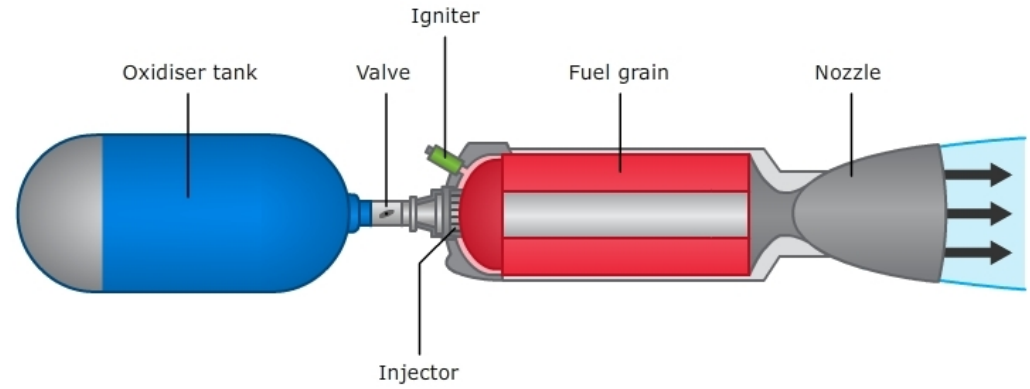
- Tank
  - Injector hole count
  - Injector total hole area
- Fuel grain:
  - Fuel inner radius
  - Fuel outer radius
- Combustion chamber and nozzle:
  - Throat length
  - Throat radius
  - Converging throat section angle
  - Nozzle length

  - Inner liner thickness
  - Pre-combustion chamber length
  - Post-combustion chamber length
  - Chamber inner radius
  - Chamber outer radius

  (I want to specify all of these in a generic manner as $f(z)$)
- Thrust
- Burn time

Output variables:

- Specific impulse



© Copyright. 2011. University of Waikato. All Rights Reserved.

Figure 1: Design of a hybrid rocket engine

2.1 Theory

2.1.1 Notation

h altitude above sea level

$\mathcal{A}$ attitude, represented by either a 3-vector of Eulerian angles or a quaternion

$(\cdot)^*, (\cdot)_{\mathbf{t}}$ value of a variable at the throat of the convergent-divergent nozzle

$(\cdot)_{\mathbf{c}}$ value of a variable inside the combustion chamber at the entrance of the nozzle

$(\cdot)_{\mathbf{e}}$ value of a variable at the exit of the nozzle

$(\cdot)_{\mathbf{amb}}$ "ambient" value of a variable in the environment outside the rocket, for example P_{amb}

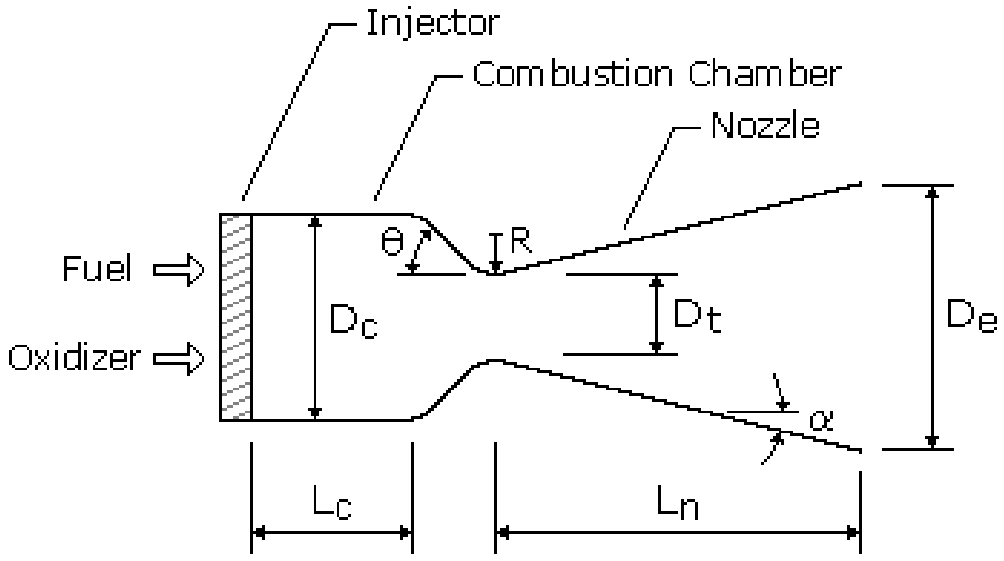


Figure 1.4

Figure 2: Design of a convergent-divergent nozzle

2.1.2 General

Ballpark of the total rocket structural mass: ~60 kg

2D attitude lookup table: $\mathcal{A} : \mathbb{R}^3, \mathbb{R}^3 \to \mathbb{R}^3 | \mathbb{H}$, $\mathcal{A}(\mathbf{x}, \mathbf{v})$

Ideal rocket equation:

$$\Delta v = v_e \ln \frac{m_\mathrm{i}}{m_\mathrm{f}} = I_\mathrm{sp} g_0 \ln \frac{m_\mathrm{i}}{m_\mathrm{f}}$$

Thrust equations:

$$\dot{m} = \rho_\mathrm{e} u_\mathrm{e} A_\mathrm{e}$$

$$T = \dot{m} u_\mathrm{e} + (P_\mathrm{e} - P_\mathrm{a}) A_\mathrm{e} = \dot{m} u_\mathrm{e} + P_\mathrm{e} A_\mathrm{e} - P_\mathrm{a} A_\mathrm{e} = T_\mathrm{vac} - (P_\mathrm{a} \cdot A_\mathrm{e})$$

Total force acting on the rocket:

$$F = T - D$$

(thrust - drag)

References:

`https://www.grc.nasa.gov/WWW/k-12/airplane/rktthsum.html`

`https://rrs.org/2023/01/28/making-correct-parabolic-nozzles/`

2.1.3 Nonlinearity

In OpenMDAO, each component maps its inputs to its outputs, $y = f(x)$. When components connect in a chain, OpenMDAO simply evaluates them sequentially. Components taken together in a group form a system of equations with the internal equations of their components.

In the `PropulsionGroup` component group, the components form a closed loop, and the system of equations happens to be non-linear. Explicitly, it consists of the following:

1. Injector flow (Bernoulli orifice)

$$\dot{m}_{\text{ox}} = C_{\text{d}} A_{\text{inj}} \sqrt{2\rho_{\text{ox}}(P_{\text{tank}} - P_{\text{c}})}$$

2. Chamber chemistry (Gibbs free energy equilibrium)

$$c^* = f_{\text{CEA}}(P_{\text{c}}, \text{MR})$$

3. Nozzle choking (1D isentropic throat flow)

$$P_{\text{c}} = \frac{\dot{m}_{\text{prop}} \cdot c^*}{A_{\text{t}}} = \frac{\dot{m}_{\text{ox}} \left(1 - \frac{1}{\text{MR}}\right) c^*}{A_{\text{t}}}$$

Substituting (1) and (2) into (3) we obtain the following master equation:

$$P_{\text{c}} = \left[\frac{C_{\text{d}} A_{\text{inj}} \left(1 + \frac{1}{\text{MR}}\right)}{A_{\text{t}}} \right] \cdot \sqrt{2\rho_{\text{ox}}(P_{\text{tank}} - P_{\text{c}})} \cdot c^*(P_{\text{c}}, \text{MR})$$

This system is nonlinear and cannot be solved in closed form for three reasons:

1. The pressure drop $\Delta P = P_{\text{tnk}} - P_{\text{c}}$ sits inside a square root
2. $c^*(P_{\text{c}}, \text{MR})$ is not constant, high pressure would suppress molecular dissociation, which changes gas molecular weight and temperature nonlinearly
3. Chamber pressure P_{c} appears both under the square root and inside the CEA chemistry table, creating an implicit self-dependency

To solve this, define the *residual function* $R(P_{\text{c}}) = 0$:

$$R(P_{\text{c}}) = P_{\text{c}} - \left[\frac{C_{\text{d}} A_{\text{inj}} \left(1 + \frac{1}{\text{MR}}\right)}{A_{\text{t}}} \right] \cdot \sqrt{2\rho_{\text{ox}}(P_{\text{tank}} - P_{\text{c}})} \cdot c^*(P_{\text{c}}, \text{MR}) = 0$$

OpenMDAO's `NewtonSolver` solves $R(P_{\text{c}}) = 0$ using an initial guess and tangent lines:

$$P_{\text{c}}^{(k+1)} = P_{\text{c}}^{(k)} - \frac{R(P_{\text{c}}^{(k)})}{\mathrm{d}R/\mathrm{d}P_{\text{c}}}$$

2.1.4 Tank

~~60~~ 70 bar tank pressure

Should be roughly 30-40 kg

2.1.5 Plumbing/injector

~30 bar pressure loss between tank and fuel grain

Either pressure drop or mass flow rate across the injector

2.1.6 Chamber/nozzle

In the following, let the asterisk superscript denote the value of a function of the vertical position z at the throat.

Area-mach relation:

$$\frac{A(z)}{A^*} = \frac{1}{M(z)}\left[\frac{2}{\gamma+1}\left(1+\frac{\gamma-1}{2}M^2(z)\right)\right]^{\frac{1}{2}\frac{\gamma+1}{\gamma-1}}$$

where A^* is the throat area, $M(z)$ is the Mach number at coordinate z.

Mass flow rate:

$$\dot{m} = A(z)P(z)\sqrt{\frac{\gamma}{TR}}M(z)\left[1+\frac{\gamma-1}{2}M^2(z)\right]^{-\frac{1}{2}\frac{\gamma+1}{\gamma-1}}$$

where T is the chamber temperature, R is the ideal gas constant, γ the specific heat ratio. For maximum flow rate, set $M(z^*) = 1$:

$$\dot{m} = A^*P^*\sqrt{\frac{\gamma}{TR}}\left[\frac{\gamma+1}{2}\right]^{-\frac{1}{2}\frac{\gamma+1}{\gamma-1}}$$

This is then used to calculate thrust according to the previous formula

References:

`https://web.stanford.edu/~cantwell/AA284A_Course_Material/Karabeyoglu%20AA%20284A%20Lectures/AA284a_Lecture2.pdf`

`https://www.grc.nasa.gov/WWW/K-12/airplane/astar.html`

`https://www.grc.nasa.gov/WWW/K-12/airplane/isentrop.html`

`https://rrs.org/2023/01/28/making-correct-parabolic-nozzles/`

2.1.7 Injector-nozzle balance

Rearrange the master equation in the following form:

$$\frac{P_\mathrm{c}}{\sqrt{P_\mathrm{tank}-P_\mathrm{c}}} = \left[\left(1+\frac{1}{\mathrm{MR}}\right)c^*C_\mathrm{d}\sqrt{2\rho_\mathrm{ox}}\right]\cdot\left(\frac{A_\mathrm{inj}}{A_\mathrm{t}}\right)$$

For given fluid densities and tank pressure, the chamber pressure depends only on the ratio of area and throat areas, $A_\mathrm{inj}/A_\mathrm{t}$. Once that ratio is fixed, scaling A_t scales total mass flow and thrust, $\dot{m}_\mathrm{prop} \propto A_\mathrm{t}$, $F = C_F P_\mathrm{c} A_\mathrm{t}$. So we can effectively use A_t to control the vehicle thrust and TWR.

C_F - dimensionless thrust coefficient. Ideal equation:

$$C_F = \sqrt{\frac{2\gamma^2}{\gamma-1}\left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{\gamma-1}}\left[1-\left(\frac{P_\mathrm{e}}{P_\mathrm{c}}\right)\right]^{\frac{\gamma-1}{\gamma}}} + \left(\frac{P_\mathrm{e}-P_\mathrm{a}}{P_\mathrm{c}}\right)\epsilon, \quad \epsilon = \frac{A_\mathrm{e}}{A_\mathrm{t}}$$

2.2 The CEA rocket solver

The CEA rocket solver is a thermodynamic state-finder that follows a parcel of gas as it moves from the chamber to the exit assuming an isentropic path (line of constant entropy). The solver always calculates at least two points: the chamber and the throat. All points specified in the parameters by the user will then be appended to the list of snapshot points.

Consider packaging the incoming reactant mixture as follows:

```
mixture = [
    { name: "H2(L)", temp: 20.27, fraction: 1.0, fuel_weight: 1.0, oxidant_weight: 0.0 },
    { name: "O2(L)", temp: 90.17, fraction: 1.0, fuel_weight: 0.0, oxidant_weight: 0.1 },
]
```

Isolating the essential arguments of the CEA rocket solver, one can define a function of the following form:

```
def solve(
    mixture : dict,        # mixture component dictionary
    pc      : float,       # chamber pressure
    subar   : list[float], # subsonic area ratio
    supar   : list[float]  # supersonic area ratio
) -> cea.RocketSolution:

    ... # implementation details

    solver.solve(solution, weights, pc, pi_p, subar=subar, supar=supar, hc)

    return solution
```

`mixture` is some kind of table packaging together the chemical components of the fuel and oxidizer, their temperatures

`pc` being the chamber pressure, a design variable

`subar` being the subsonic area ratio, A_c/A^*

`supar` being the supersonic area ratio, A_e/A^*

`pi_p` being the pressure ratio P_c/P_e of chamber pressure to exit pressure. The reactants are ideally expanded at $P_e = P_{amb}$, however since the atmospheric conditions change during the flight, the ideal expansion condition

CEA computes *intensive* properties (values per unit mass), to get *extensive* properties (N and kg/s), we provide the scale in terms of the throat area A^*.

Mass flow:

$$\dot{m} = \frac{PA^*}{c^*}$$

where P is the chamber pressure and c^* is the *characteristic velocity* computed with CEA.

Thrust:

$$T = \dot{m} I_{sp} g_0 = C_f A^* P$$

compute with I_{sp} or C_f (thrust coefficient) provided by CEA.

References:

`https://nasa.github.io/cea/examples/rocket/example8.html`

2.3 My own set of inputs

Flight plan: attitude as a function of altitude and velocity, or pitch as a function of velocity

Aerodynamic

Mass and CoG time-series (CoG from mass flow?)

Chemical composition time-series -> (gamma, M_w, T) change

Nozzle erosion?

—

A(x,v)

C_D(v, atm)

M_0 (~25.0 kg total) M_f M_O

—

P_T = 60 bar

P_c = 30 bar

P_{inj} > 30 bar

NOTE: optimizing for sea-level thrust

Regression rate